

analysis (Python)



Import notebook

```
# MAGIC %md
# MAGIC # PySpark Data Processing Pipeline: NYC Taxi Data
# MAGIC
# MAGIC This notebook implements a full data processing pipeline using PySpark on the built-in NYC Taxi
dataset.
# MAGIC ---
```

```
# MAGIC %md
# MAGIC ## Step 1: Setup and Data Loading
# MAGIC
# MAGIC We'll load the main taxi trip data (yellow_tripdata) and the taxi zone lookup table. These are
available in the Databricks sample datasets.
```

```
from pyspark.sql.functions import col, year, month, dayofmonth, round, avg, count, sum, lit, to_timestamp

trip_data_path = "/databricks-datasets/nyctaxi/tripdata/yellow/yellow_tripdata_2019-01.csv.gz"
zone_lookup_path = "/databricks-datasets/nyctaxi/taxizone/taxi_zone_lookup.csv"

# Output path for our processed data
output_path = "/tmp/NYCTaxiPipeline/analysis_results/"
```

```
# Load Trip Data
# We'll let Spark infer the schema and handle the header
raw_trips_df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load(trip_data_path)

zone_lookup_df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load(zone_lookup_path)

print("Data loaded successfully.")
```

```
▶ raw_trips_df: pyspark.sql.connect.dataframe.DataFrame = [VendorID: integer, tpep_pickup_datetime: timestamp ... 16 more fields]
▶ zone_lookup_df: pyspark.sql.connect.dataframe.DataFrame = [LocationID: integer, Borough: string ... 2 more fields]
```

Data loaded successfully.

```
# MAGIC %md
# MAGIC ## Step 2: Data Processing Pipeline (DataFrame API)
# MAGIC
# MAGIC We will now apply a series of transformations.
# MAGIC
# MAGIC **Optimization:** We apply filters **as early as possible** to reduce the amount of data processed in
later stages.
```

```

;

# Transformation 1: Apply Early Filters (2+ filters)
# Filter out nulls and trips with no passengers or zero distance
filtered_trips_df = raw_trips_df \
    .filter(col("passenger_count") > 0) \
    .filter(col("trip_distance") > 0) \
    .filter(col("PULocationID").isNotNull()) \
    .filter(col("DOLocationID").isNotNull())

```

▶  filtered_trips_df: pyspark.sql.connect.dataframe.DataFrame = [VendorID: integer, tpep_pickup_datetime: timestamp ... 16 more fields]

```

# Transformation 2: Column Transformations (withColumn)
# Cast timestamps and create new features
transformed_trips_df = filtered_trips_df \
    .withColumn("tpep_pickup_datetime", to_timestamp(col("tpep_pickup_datetime"))) \
    .withColumn("tpep_dropoff_datetime", to_timestamp(col("tpep_dropoff_datetime"))) \
    .withColumn("trip_duration_seconds", (col("tpep_dropoff_datetime").cast("long") -
col("tpep_pickup_datetime").cast("long"))) \
    .withColumn("trip_duration_minutes", round(col("trip_duration_seconds") / 60, 2)) \
    .withColumn("cost_per_mile", round(col("total_amount") / col("trip_distance"), 2))

```

▶  transformed_trips_df: pyspark.sql.connect.dataframe.DataFrame = [VendorID: integer, tpep_pickup_datetime: timestamp ... 19 more fields]

```

# Transformation 3: Join Operation
# We create aliases for the zone_lookup_df to avoid ambiguous columns
pickup_zones = zone_lookup_df.alias("pickup")
dropoff_zones = zone_lookup_df.alias("dropoff")

# Join trips with zones to get human-readable location names
joined_df = transformed_trips_df \
    .join(pickup_zones.withColumnRenamed("Zone", "Pickup_Zone").withColumnRenamed("Borough",
"Pickup_Borough"),
        col("PULocationID") == col("pickup.LocationID"), # Use aliased column
        "left") \
    .join(dropoff_zones.withColumnRenamed("Zone", "Dropoff_Zone").withColumnRenamed("Borough",
"Dropoff_Borough"),
        col("DOLocationID") == col("dropoff.LocationID"), # Use aliased column
        "left")

# Prune columns to keep only what we need (another optimization)
analysis_df = joined_df.select(
    "tpep_pickup_datetime",
    "passenger_count",
    "trip_distance",
    "total_amount",
    "payment_type",
    "trip_duration_minutes",
    "cost_per_mile",
    "Pickup_Zone",
    "Pickup_Borough",
    "Dropoff_Zone",
    "Dropoff_Borough"
)

```

▶  analysis_df: pyspark.sql.connect.dataframe.DataFrame = [tpep_pickup_datetime: timestamp, passenger_count: integer ... 9 more fields]
▶  dropoff_zones: pyspark.sql.connect.dataframe.DataFrame = [LocationID: integer, Borough: string ... 2 more fields]
▶  joined_df: pyspark.sql.connect.dataframe.DataFrame = [VendorID: integer, tpep_pickup_datetime: timestamp ... 27 more fields]
▶  pickup_zones: pyspark.sql.connect.dataframe.DataFrame = [LocationID: integer, Borough: string ... 2 more fields]

```
# Transformation 4: GroupBy and Aggregation
# Finding the busiest boroughs
borough_summary_df = analysis_df \
    .groupBy("Pickup_Borough") \
    .agg(
        count("*").alias("total_trips"),
        avg("total_amount").alias("avg_fare"),
        avg("trip_distance").alias("avg_distance")
    ) \
    .orderBy(col("total_trips").desc())
```

►  borough_summary_df: pyspark.sql.connect.dataframe.DataFrame = [Pickup_Borough: string, total_trips: long ... 2 more fields]

```
# Action 1: Show the result of the aggregation
print("Borough Summary:")
borough_summary_df.show()
```

Borough Summary:

Pickup_Borough	total_trips	avg_fare	avg_distance
Manhattan	6807339	13.507881991004613	2.2225421974723334
Queens	446545	45.0263214010611	11.59962187461474
Unknown	146149	15.418409978866686	2.5998101937064173
Brooklyn	82588	19.753469632409633	4.517867850050868
Bronx	14322	26.06881092025827	6.943785784108348
Staten Island	292	50.37804794520529	13.4234589041096
EWB	145	83.89599999999992	7.477448275862066

```
# MAGIC %md
# MAGIC ## Step 3: SQL Queries
# MAGIC
# MAGIC We'll register our main `analysis_df` as a temporary view to run SQL queries.
```

```
# Register the DataFrame as a temporary SQL view
analysis_df.createOrReplaceTempView("taxi_trips_view")
```

```
%sql
-- SQL Query 1: Average fare by payment type
SELECT
    CASE
        WHEN payment_type = 1 THEN 'Credit Card'
        WHEN payment_type = 2 THEN 'Cash'
        WHEN payment_type = 3 THEN 'No Charge'
        WHEN payment_type = 4 THEN 'Dispute'
        ELSE 'Unknown/NA'
    END AS payment_method,
    COUNT(*) AS num_trips,
    AVG(total_amount) AS avg_fare
FROM
    workspace.default.nyc_taxi_analysis
GROUP BY
    payment_type
ORDER BY
    num_trips DESC;
```

▸  _sqldf: pyspark.sql.connect.dataframe.DataFrame = [payment_method: string, num_trips: long ... 1 more field]

Table

 This result is stored as _sqldf and can be used in other Python and SQL cells.

```
%sql
-- SQL Query 2: Top 10 Busiest Pickup Zones
SELECT
  Pickup_Zone,
  Pickup_Borough,
  COUNT(*) AS total_pickups
FROM
  workspace.default.nyc_taxi_analysis
WHERE
  Pickup_Zone IS NOT NULL
GROUP BY
  Pickup_Zone,
  Pickup_Borough
ORDER BY
  total_pickups DESC
LIMIT 10;
```

▸  _sqldf: pyspark.sql.connect.dataframe.DataFrame = [Pickup_Zone: string, Pickup_Borough: string ... 1 more field]

Table

 This result is stored as _sqldf and can be used in other Python and SQL cells.

```
# MAGIC %md
# MAGIC ## Step 4: Performance Analysis
# MAGIC
# MAGIC ### 4.1 `.explain()` Plan
# MAGIC
# MAGIC Checking the physical plan for our `borough_summary_df` aggregation.
```

```
print("Physical Plan for Borough Summary:")
borough_summary_df.explain()
```

```

      :      +- PhotonProject [LocationID#14406, Borough#14407 AS Pickup
_Borough#14425]
      :      +- PhotonFilter isnotnull(LocationID#14406)
      :      +- PhotonRowToColumnar
      :      +- FileScan csv [LocationID#14406,Borough#14407] B
atched: false, DataFilters: [isnotnull(LocationID#14406)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/databricks-datasets/nyctaxi/taxizone/taxi_zone_lookup.csv], PartitionFilters: [], PushedFilters: [IsNotNull(LocationID)], ReadSchema: struct<LocationID:int,Borough:string>
      +- PhotonShuffleExchangeSource
      +- PhotonShuffleMapStage EXECUTOR_BROADCAST, [id=#23849]
      +- PhotonShuffleExchangeSink SinglePartition
      +- PhotonFilter isnotnull(LocationID#14572)
      +- PhotonRowToColumnar
      +- FileScan csv [LocationID#14572] Batched: false, DataFilters: [isnotnull(LocationID#14572)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/databricks-datasets/nyctaxi/taxizone/taxi_zone_lookup.csv], PartitionFilters: [], PushedFilters: [IsNotNull(LocationID)], ReadSchema: struct<LocationID:int>

== Photon Explanation ==
The query is fully supported by Photon.
```

```
# MAGIC %md
# MAGIC ### 4.2 Spark UI and Query Details
# MAGIC
```

```
# Action 2: Write results to a Delta Table
# This uses saveAsTable for persistence, which is a better practice in Databricks
# and avoids low-level DBFS file path issues.

print("Writing final data to Delta Lake table 'nyc_taxi_analysis'...")

# Note: We are using a temporary table name. This will be visible in the Data Explorer.
analysis_df.write \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .partitionBy("Pickup_Borough") \
    .saveAsTable("nyc_taxi_analysis")

print("Write complete. Data is now saved as a partitioned Delta table named 'nyc_taxi_analysis'.")
```

```
Writing final data to Delta Lake table 'nyc_taxi_analysis'...
Write complete. Data is now saved as a partitioned Delta table named 'nyc_taxi_analysis'.
```

```
import time

start_time = time.time()
count_1 = analysis_df.count()
end_time = time.time()
uncached_time = end_time - start_time
print(f"Uncached count: {count_1} (Took {uncached_time:.2f} seconds)")
```

```
Uncached count: 7497380 (Took 11.75 seconds)
```

```
# Cache the DataFrame -- not possible on serverless setting, using alternative
display(analysis_df)
```

Table

```
# Second run: Cached
# The .count() action will force the cache to be populated
start_time = time.time()
count_2 = analysis_df.count()
end_time = time.time()
cached_time_1 = end_time - start_time
print(f"First cached count: {count_2} (Took {cached_time_1:.2f} seconds)")
```

First cached count: 7497380 (Took 11.75 seconds)

```
# Third run: Fully cached
start_time = time.time()
count_3 = analysis_df.count()
end_time = time.time()
cached_time_2 = end_time - start_time
print(f"Second cached count: {count_3} (Took {cached_time_2:.2f} seconds)")
```

Second cached count: 7497380 (Took 11.51 seconds)

```
# MAGIC %md
# MAGIC ## Step 5: Actions vs. Transformations
# MAGIC
# MAGIC This demonstrates Spark's lazy evaluation.
```

```
# 1. Define a Transformation
# This returns a new DataFrame definition but does *not* execute a job.
print("Defining a transformation...")
highly_paid_trips = analysis_df.filter(col("total_amount") > 100)

print("Transformation defined. No job was triggered.")
print("Type:", type(highly_paid_trips))
```

►  highly_paid_trips: pyspark.sql.connect.dataframe.DataFrame = [tpep_pickup_datetime: timestamp, passenger_count: integer ... 9 more fields]

Defining a transformation...

Transformation defined. No job was triggered.

Type: <class 'pyspark.sql.connect.dataframe.DataFrame'>

```
# 2. Call an Action
# This forces Spark to execute the plan (load -> filter -> filter -> ... -> join -> filter)
print("\nCalling an action (.show())...")
highly_paid_trips.show(5)

print("Action complete. A Spark job was triggered to get this result.")
```

Calling an action (.show())...

tpep_pickup_datetime	passenger_count	trip_distance	total_amount	payment_type	trip_duration_minutes	cost_per_mile
Pickup_Zone Pickup_Borough	Dropoff_Zone Dropoff_Borough					
2019-01-01 00:23:11 NA Unknown	1 NA Unknown	0.34	185.3	1	0.23	545.0
2019-01-01 00:38:36 Gramercy Manhattan	2 NA Unknown	33.19	131.88	1	42.95	3.97
2019-01-01 00:13:17 JFK Airport Queens	1 NA Unknown	44.1	150.3	2	52.93	3.41
2019-01-01 00:22:44 NA Unknown	1 NA Unknown	0.97	151.0	1	0.1	155.67
2019-01-01 01:00:57 Union Sq Manhattan	2 NA Unknown	6.54	111.3	2	29.47	17.02

only showing top 5 rows

Action complete. A Spark job was triggered to get this result.

```
# MAGIC %md
# MAGIC
# MAGIC Building a simple linear regression model to predict `total_amount` based on `trip_distance`.
```

```

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression

# Select features for the model
# We need to filter out edge cases for a stable model
ml_df = analysis_df \
    .filter(col("total_amount") > 0) \
    .filter(col("total_amount") < 500) \
    .filter(col("trip_distance") > 0) \
    .filter(col("trip_distance") < 100) \
    .select("total_amount", "trip_distance", "passenger_count") \
    .na.drop()

# 1. Assemble features into a single vector
assembler = VectorAssembler(
    inputCols=["trip_distance", "passenger_count"],
    outputCol="features"
)
ml_data = assembler.transform(ml_df)

# 2. Split data
(training_data, test_data) = ml_data.randomSplit([0.8, 0.2], seed=42)

# 3. Define and train the model
lr = LinearRegression(featuresCol="features", labelCol="total_amount")
lr_model = lr.fit(training_data)

# 4. Print model coefficients
print(f"Model Coefficients:")
print(f"Intercept: {lr_model.intercept:.2f}")
print(f"Trip Distance: {lr_model.coefficients[0]:.2f}")
print(f"Passenger Count: {lr_model.coefficients[1]:.2f}")

# 5. Evaluate on test data
predictions = lr_model.transform(test_data)
predictions.select("total_amount", "prediction").show(5)

```

▶  ml_data: pyspark.sql.connect.dataframe.DataFrame
 ▶  ml_df: pyspark.sql.connect.dataframe.DataFrame = [total_amount: double, trip_distance: double ... 1 more field]
 ▶  predictions: pyspark.sql.connect.dataframe.DataFrame
 ▶  test_data: pyspark.sql.connect.dataframe.DataFrame
 ▶  training_data: pyspark.sql.connect.dataframe.DataFrame

Model Coefficients:

Intercept: 5.97

Trip Distance: 3.37

Passenger Count: -0.02

total_amount	prediction
0.3	6.288730200177323
0.3	6.288730200177323
0.3	6.288730200177323
0.3	6.288730200177323
0.3	6.288730200177323

only showing top 5 rows

